Homework Assignment 2

Agent-Based Models and Graph Theory -- a vending-machine ecosystem

In Homework 1 (Question 4.2) you imagined how your `VendingMachine` class might live inside an **agent-based model (ABM)**. In this assignment you will actually build one: customers who get hungry and buy snacks from vending machines, and then you will analyze the physical layout of those machines as a **graph**. Along the way you will practice **classes, inheritance, and composition** (Days 6-7), **agent-based modeling** (Days 8-9), and **graph theory with `networkx`** (Days 10-11).

✅ Matthew Hoppenrath

Goals for this homework assignment

By the end of this assignment you should be able to:

- Build and use Python **classes**, including **inheritance** (subclasses) and **composition** (a class that holds and coordinates other objects).
- Implement a simple **agent-based model**: give agents state and rules, advance the system in time, and **interpret** the emergent behavior.
- Represent real-world data as a **graph** with `networkx`, visualize it, and compute basic structural properties (degree, neighbors, subgraphs, graph coloring).
- Keep your work under **version control** with Git.

Work through the following assignment, making sure to follow all of the directions and answer all of the questions.

There are **100 points** possible on this assignment. Point values for each part are included in the section headers and question prompts.

This assignment is due at 11:59 pm on Friday, October 30. You will submit it through **Gradescope** this semester (not D2L): you will upload your completed notebook, an autograder converts it to a PDF, and you then map the PDF pages to the questions for grading. Follow the step-by-step Gradescope Submission Instructions on the course website (also linked at the end of this notebook).

How this connects to the course learning goals

Beyond the specific goals above, this homework builds toward several **course-level learning goals** (see the syllabus or the Day 1 notebook for the full CLG 1-7 list):

- **CLG 1 - Design and write programs to solve problems:** you'll design and build an agent-based model from several interacting classes (Part 2) and construct a graph of the vending-machine network (Part 3).
- **CLG 3 - Choose appropriate data types:** you'll use **inheritance** and **composition** to structure the model, and represent the campus layout as a `networkx` graph (nodes, edges, and attributes).
- **CLG 2 - Identify and use pre-existing tools:** you'll lean on `networkx` (plus `pandas` / `matplotlib`), reading its documentation to find the right built-in methods, and transparently document any AI-tool use (Part 4).
- **CLG 4 - Use modern software development tools:** you'll use Git (add/commit/push) throughout, with incremental per-part commits, and write reusable class code.
- **CLG 5 - Manipulate, analyze, and visualize datasets:** you'll run and plot the simulation, and analyze and visualize the machine-network graph.
- **CLG 7 - Communicate and explain your work in writing:** you'll interpret your simulation and graph results, justify your design choices, and reflect on your process and resource/AI use (Part 4).

Table of contents

1. Part 1: Git setup (4 points)
2. Part 2: Vending-machine agent-based model (46 points)
3. Part 3: Machine-map graph theory (40 points)
4. Part 4: Reflection and AI/resource audit (10 points)

Each part ends with a 1-point checkpoint where you commit your work to Git.

Part 1: Add this assignment to your Git repository (4 points)

Before you get too far, add this notebook to your `cmse202-f26-turnin` repository so your progress is tracked and your final version is preserved. (Make sure your instructor and TA are collaborators on that repository -- you set this up for Homework 01.) **If this notebook is not inside your turnin repo, none of your changes will be tracked**, so work in the copy that lives there.

✅ **Question 1.1 (4 points):** In a terminal, **clone** your `cmse202-f26-turnin` repository (if you do not already have a local copy), create a new directory inside it called `hw-02`, move/copy this notebook into that directory, and then **add**, **commit**, and **push** it. In the cell below, record (as comments) **all** the commands you used: cloning the repo, **creating the `hw-02` directory**, **moving/copying this notebook into it**, and the `add`, `commit`, and `push`.

In [ ]:
```
# Put the git commands you used here (as comments):
# clone, create the hw-02 directory, move the notebook in, then add, commit, and pu
#git clone https://github.com/matthoppy07-dev/CMSE-202-f26-turnin.git
#cd CMSE-202-f26-turnin
#mkdir hw-02
#cd hw-02
#mv HW02-ABMandGraphTheory-STUDENT.ipynb CMSE-202-f26-turnin/hw-02/
# git add HW02-ABMandGraphTheory-STUDENT.ipynb
#git commit -m "hw2"
#git push origin main
```

Part 2: Build a vending-machine agent-based model (46 points)

Imagine a venue -- say, a campus building during a busy afternoon. There are a few **vending machines** at fixed locations, and a crowd of **customers**. Over time customers get hungry; when a customer is hungry enough (and has enough cash), they walk to the **nearest** vending machine and try to buy a snack.

You are **provided** with a working `VendingMachine` class below -- a cleaned-up, extended version of the one you built in Homework 1 (its methods now use standard Python `snake_case` naming). **Do not modify it.** Your job is to build the *agents* (the customers) and the *environment* (the venue) around it.

In [76]:
```
## DO NOT MODIFY THIS CELL -- this is your provided VendingMachine class.
class VendingMachine:
    def __init__(self, starting_treats, location, price=1):
        self.inventory = list(starting_treats)   # list of snack names
        self.stash = 0                           # money collected ($)
        self.current_transaction = 0             # cash inserted but not yet spent
        self.price = price                       # price of one snack ($)
        self.location = location                 # (x, y) position of the machine

    def restock(self, new_list):
        for item in new_list:
            self.inventory.append(item)

    def is_available(self, item):
        return item in self.inventory
```

```
    def input_cash(self, cash):
        self.current_transaction += cash

    def vend(self, item):
        self.inventory.remove(item)

    def make_request(self, item):
        """If enough cash has been inserted and the item is available, vend it.
        Returns True on a successful purchase, False otherwise."""
        if self.current_transaction >= self.price and self.is_available(item):
            self.vend(item)
            self.stash += self.price
            self.current_transaction -= self.price
            return True
        return False
```

✅ **Question 2.1 (10 points):** Complete the `Customer` base class below so that:

- `__init__(self, cash, position, hunger_rate=1.0, hunger_threshold=3)` stores each argument as an attribute of the same name, and also creates a `hunger` attribute that starts at **0**. Watch the types: `position` is an `(x, y)` **tuple**, while `cash`, `hunger_rate`, and `hunger_threshold` are plain **numbers**.
- `get_hungrier(self)` increases `self.hunger` by `self.hunger_rate`.
- `wants_to_buy(self)` returns `True` if the customer is hungry enough (`hunger >= hunger_threshold`) **and** has at least $1 of cash; otherwise `False`.

Give each method a short **docstring** describing what it does.

> **Reminder -- how to write a docstring:** a docstring is a short text description placed as the **first line inside** a function or method, written with **triple quotes**. For example:
>
> ```
> def add(a, b):
>     """Return the sum of a and b."""
>     return a + b
> ```

Your docstring can be a single short sentence -- just enough to say what the method does. It's fine if it closely echoes the comment already in the starter cell; the goal here is simply to practice writing docstrings.

In [68]:
```
# Complete the Customer class.
class Customer:
    def __init__(self, cash, position, hunger_rate=1.0, hunger_threshold=3):
        """starts the customer with money, location, and hunger"""
        self.cash = cash
        self.position = position
        self.hunger_rate = hunger_rate
        self.hunger_threshold = hunger_threshold
        self.hunger = 0
```

```
    def get_hungrier(self):
        """Makes hunger level rise at hunger rate"""
        self.hunger += self.hunger_rate

    def wants_to_buy(self):
        """Makes sure the customer is hungry enough for food and has enough money""
        if self.hunger >= self.hunger_threshold and self.cash >= 1:
            return True
        else:
            return False
```

✅ **Question 2.2 (8 points):** Using **inheritance**, create **two** subclasses of `Customer` that represent different kinds of patrons. Name them `Student` and `Visitor` (so the test and setup cells below work).

- **Each subclass needs its own `__init__(self, position)`** that calls the parent constructor via `super().__init__(...)`, passing cash and hunger settings **you** choose. The only hard requirement is that `Visitor`s carry **more cash** than `Student`s.
- The two must differ in **behavior** -- and, importantly, that difference has to be something the simulation will actually *see* when it runs. `Venue.step` (which you write next) only ever calls a customer's `get_hungrier()` and `wants_to_buy()`, so make your difference show up **through those methods**, in one (or both) of these ways:
    1. give the subclasses **different values** for the attributes those methods use (`cash`, `hunger_rate`, `hunger_threshold`) -- e.g. an *impulsive* `Student` who gets hungry fast (a larger `hunger_rate`) and buys at a low threshold, versus a *patient* `Visitor` who gets hungry slowly; **and/or**
    2. **override** `wants_to_buy` (or `get_hungrier`) in a subclass -- e.g. a *thrifty* `Visitor` who refuses to spend their last dollar, keeping a couple of dollars in reserve.

> ⚠️ A brand-new method, or an attribute that nothing reads, **won't change the simulation on its own** -- `step` will never call it. Save ideas like customers **moving around**, **snack preferences**, or machines **restocking** for **Question 2.6**, where you control the step loop and can wire in new behavior yourself.

Then, in the markdown cell that follows your code, **describe the two personalities you created** and the behavioral difference between them (1-2 sentences each).

In [71]:
```
# Define the Student and Visitor subclasses.
class Student(Customer):
    def __init__(self, position):
        """The student has less cash and gets hungry more often, which means a lowe
        Customer.__init__(self,cash = 10, position = position,hunger_rate = 1, hung
```

```
class Visitor(Customer):
    def __init__(self, position):
        """The visitor has more cash, get hungry less often, which means a higher t
        Customer.__init__(self, cash=20, position=position, hunger_rate=1.0, hunger
```

*Student:The student has less cash and gets hungry more often, which means a lower threshold as well. Visitor: The visitor has more cash, get hungry less often, which means a higher threshold. *

✅ **Question 2.3 (12 points):** Complete the `Venue` class, which **composes** the machines and customers and runs the simulation. It should have:

- `__init__(self, machines, customers)` -- store the list of `machines` and the list of `customers`, and create an empty list `self.history` (you will record one summary number per step here -- you decide what it measures in Question 2.4).
- `nearest_machine(self, customer)` -- return the `VendingMachine` in `self.machines` whose `location` is closest to the customer's `position`. Use the **straight-line (Euclidean) distance** between the customer at (x_1, y_1) and a machine at (x_2, y_2): $$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$ (Python's `math.hypot(x2 - x1, y2 - y1)` computes exactly this, or you can write it out yourself.)
- `step(self)` -- advance the simulation by one time step:
    1. For **each** customer: call `get_hungrier()`.
    2. If that customer `wants_to_buy()`, find their `nearest_machine`. If that machine still has inventory and the customer can afford the price, the customer buys one snack: insert the price with `input_cash`, call `make_request` for any item currently in the machine's `inventory` (for example the first one, `machine.inventory[0]` or any other valid index), and if the purchase succeeds, subtract the price from the customer's `cash` and reset their `hunger` to 0.
    3. After all customers have acted, append **one summary number for this step** to `self.history`. This is the quantity you will study in Question 2.4 -- for example the **total revenue** so far, the **total snacks sold**, or **how many customers are still hungry**. Pick one and record it.
- `run(self, n_steps)` -- call `step()` `n_steps` times and return `self.history`.

In [74]:
```
# Complete the Venue class.
import math
```

```
class Venue:
    def __init__(self, machines, customers):
        """Initialize the venue with the vending the customers and the simulation l
        self.machines = machines
        self.customers = customers
        self.history = []

    def nearest_machine(self, customer):
        """Finds the closest machine to a customer"""
        closemachine = self.machines[0]
        for i in self.machines:
            if math.dist(i.location, customer.position) < math.dist(closemachine.lo
                closemachine = i
        return closemachine

    def step(self):
        for i in self.customers:
            i.get_hungrier()
            if i.wants_to_buy():
                machine = self.nearest_machine(i)
                if len(machine.inventory) > 0 and i.cash >= machine.price:
                    item = machine.inventory[0]
                    machine.input_cash(machine.price)
                    bought = machine.make_request(item)
                    if machine.make_request(item):
                        i.cash -= machine.price
                        i.hunger = 0
        totrev = 0
        for j in self.machines:
            totrev = totrev + j.stash
        self.history.append(totrev)

    def run(self, n_steps):
        """ runs simulation for a set number of steps"""
        for k in range(n_steps):
            self.step()
        return self.history
```

Before moving on, run the cell below to **check your classes**. This is not a graded question -- it is a quick self-test: it builds a few customers and machines and confirms that your `Customer`, `Student` / `Visitor`, and `Venue` behave the way Questions 2.1-2.3 describe (for example, that a `Visitor` really does carry more cash than a `Student`, that `nearest_machine` returns the closest machine, and that a hungry, well-funded customer actually buys a snack in one step). If it prints `All tests passed!` with no error, your classes are working. If it raises an error, the assertion that failed tells you which behavior to go back and fix. **Do not modify this cell.**

In [82]:
```
# testing -- do not change this cell!
# If your classes are correct, this cell runs with no errors and prints "All tests
c = Customer(cash=5, position=(0, 0))
assert c.cash == 5 and c.hunger == 0
c.get_hungrier()
```

```
assert c.hunger > 0

s = Student(position=(0, 0))
v = Visitor(position=(0, 0))
assert isinstance(s, Customer) and isinstance(v, Customer)   # inheritance
assert v.cash > s.cash                                       # visitors carry more

m1 = VendingMachine(['Twix'], location=(0, 0))
m2 = VendingMachine(['Twix'], location=(10, 10))
assert Venue([m1, m2], []).nearest_machine(Customer(cash=5, position=(1, 1))) is m1

hungry = Customer(cash=5, position=(0, 0), hunger_rate=10)   # exceeds threshold in
test_venue = Venue([VendingMachine(['Twix', 'Twix'], location=(0, 0))], [hungry])
before = len(test_venue.machines[0].inventory)
test_venue.step()
assert len(test_venue.machines[0].inventory) == before - 1   # one snack sold
assert test_venue.machines[0].stash == 1
print("All tests passed!")
```

```
All tests passed!
```

With your classes working, let's set up the simulation. Rather than a single fixed crowd, the cell below (run it **as-is, without modifying it**) gives you a helper, `build_venue(n_students, n_visitors)`, that returns a fresh `Venue`: **three** freshly stocked vending machines at fixed locations, plus the mix of `Student`s and `Visitor`s you ask for, dropped at random spots across a 10x10 area. The random seed is fixed, so the customers land in the **same spots** every time -- which lets you compare different *populations* fairly. The cell also builds one ready-to-use mixed venue called `venue` (8 `Student`s + 8 `Visitor`s). You'll use the helper in Question 2.4 to see whether your `Student`s and `Visitor`s actually behave differently.

In [83]:
```
## DO NOT MODIFY THIS CELL -- it sets up the simulation.
import random

SNACKS = ['Twix', 'Snickers', 'M&Ms', 'Chips', 'Pretzels']

def build_venue(n_students, n_visitors, seed=202610):
    """Build a fresh Venue: three freshly-stocked machines plus the requested mix o
    The seed is fixed so customer positions are identical across populations (a fai
    rng = random.Random(seed)
    machines = [VendingMachine([s for s in SNACKS for _ in range(4)], location=loc)
                for loc in [(1, 1), (8, 8), (1, 8)]]
    customers  = [Student(position=(rng.randint(0, 9), rng.randint(0, 9))) for _ in
    customers += [Visitor(position=(rng.randint(0, 9), rng.randint(0, 9))) for _ in
    return Venue(machines, customers)

venue = build_venue(8, 8)   # a ready-made mixed crowd (8 students + 8 visitors)
print("Setup ready: build_venue(n_students, n_visitors) is defined, and `venue` hol
      len(venue.customers), "customers across", len(venue.machines), "machines.")
```

```
Setup ready: build_venue(n_students, n_visitors) is defined, and `venue` holds a mix
ed crowd of 16 customers across 3 machines.
```

✅ **Question 2.4 (6 points):** First decide **which outcome** your model should track over time -- the summary number your `Venue.step` records in `self.history` (for example total revenue, total snacks sold, or the number of customers still hungry). Then use `build_venue` to create **three** venues with the **same total number of customers (16)** but different populations:

- **all `Student`s** -- `build_venue(16, 0)`,
- **all `Visitor`s** -- `build_venue(0, 16)`, and
- a **mixed** crowd -- `build_venue(8, 8)` (this is the `venue` already built for you).

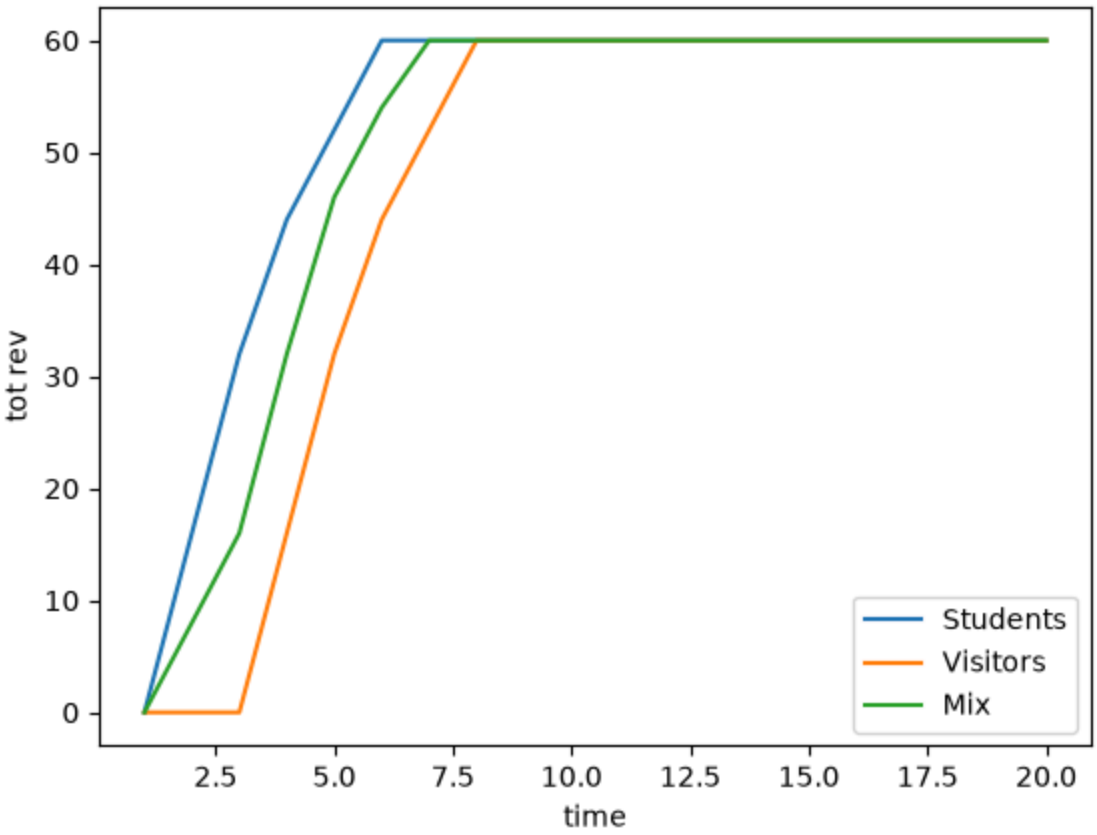
Run each for 20 time steps and plot all **three** histories on a **single** labeled line plot (axis labels, a **legend** identifying the three populations, and a title). In a comment, state **which outcome you chose and why it is interesting**.

In [84]:
```
# Put your code here.
# 1) Build three venues with build_venue: all students, all visitors, and a mixed c
# 2) Run each for 20 steps and plot all three histories on ONE labeled plot (with a
# In a comment, say which outcome you are tracking and why.
import matplotlib.pyplot as plt

venuestu = build_venue(16,0)
venuevis = build_venue(0,16)
venuemix = build_venue(8,8)

steps = 20
totstu = venuestu.run(steps)
totvis= venuevis.run(steps)
totmix = venuemix.run(steps)

time = list(range(1,steps+1))
plt.plot(time, totstu, label = "Students")
plt.plot(time,totvis, label = "Visitors")
plt.plot(time,totmix, label = "Mix")
plt.xlabel("time")
plt.ylabel("tot rev")
plt.legend()
plt.show()
```



✅ **Question 2.5 (4 points):** In the cell below, **interpret** your results:

- Looking at your Q2.4 plot, how do the **three populations differ**, and what does that tell you about how your `Student`s and `Visitor`s behave? (If the three curves came out essentially *identical*, that's a sign your subclass difference isn't reaching `step` -- go back and revisit Q2.2.)
- What does the **shape** of a curve (for example, rising and then leveling off) tell you about what is happening in the venue over time?
- Name **one** thing you could change about the model (more machines, more frequent restocking, different machine placement, customer behavior, ...) that you think would improve the outcome you tracked, and briefly say why. Also note **one limitation** of this model as a description of a real venue.

✏️ *Visitors take longer to start spending, whereas students and mix are almost instantly spending. Then as time progresses visitors have a much higher slope, whereas a mix and students have less of a slope and start to even out. In general the shape Looks like a logrithmic graph for all three but it could be because they all ran out of food in the machine. We could map it out like in a mall so we can add more features like lines and other things, this would make it more realistic, but it would be difficult to shape the things around the vending machines and animate the people. *

✅ **Question 2.6 (5 points):** Now make the model **your own**. Add **one feature** that makes it more realistic. Some ideas (pick one, or invent your own):

- machines that get **restocked** every few steps,
- customers that **move** between steps,
- machines that occasionally **break** and stop vending,
- **prices that vary** from machine to machine,
- customers with **snack preferences**.

Implement your feature by **subclassing or editing your own classes** (`Customer` , a subclass, or `Venue`) -- **do not modify the provided** `VendingMachine` . Then **run the model again** and make a short plot or printout showing the feature's effect. In the markdown cell, **describe the feature you added and how it changed the results.**

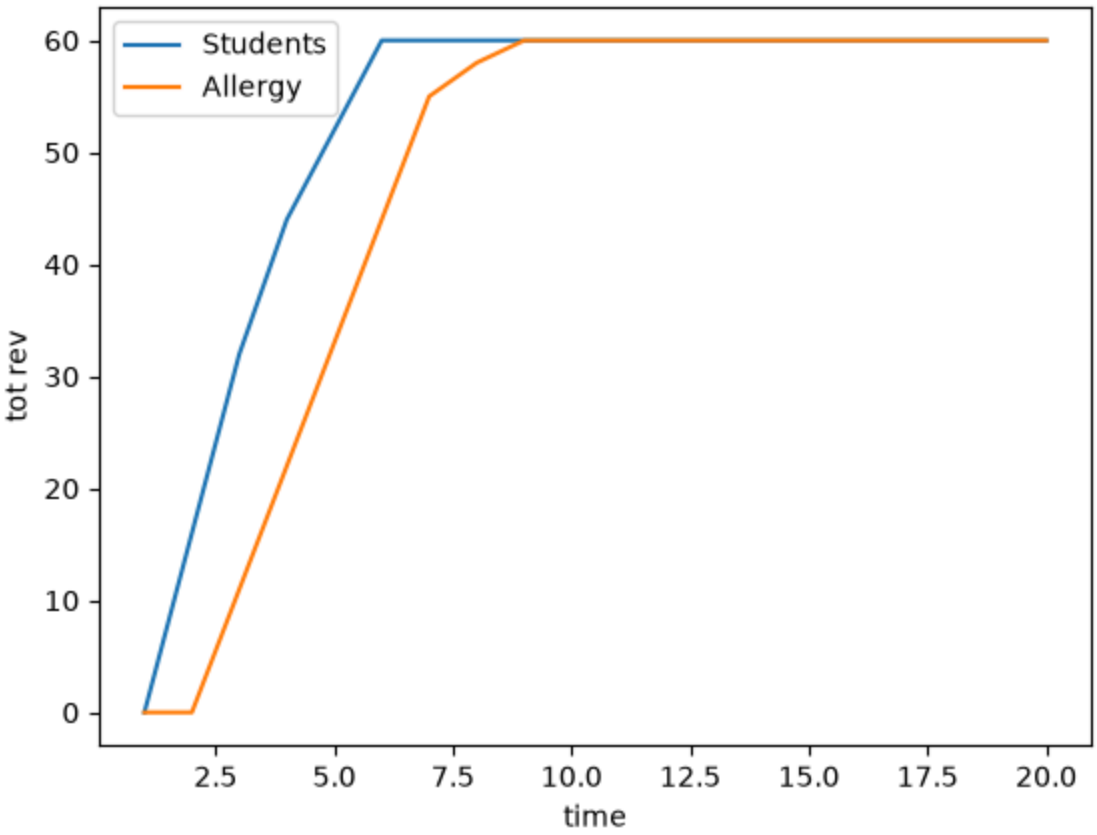
In [85]:
```
# Put your enhancement here: implement the feature, run the model, and show its eff
class Allergy(Customer):
    def __init__(self,position, allergy_rate = .3):
        Customer.__init__(self,15,position,hunger_rate = 1, hunger_threshold = 3)
        self.is_allergic = random.random() < allergy_rate

    def wants_to_buy(self):
        if self.is_allergic:
            return False
        return self.hunger>= self.hunger_threshold and self.cash >=1
```

In [86]:
```
venuestu = build_venue(16,0)
base = build_venue(16,0)
allg_customers = []
for i in base.customers:
    allg_customers.append(Allergy(i.position))
venueallg = Venue(base.machines, allg_customers)

steps = 20
totstu = venuestu.run(steps)
totallg = venueallg.run(steps)

time = list(range(1,steps+1))
plt.plot(time, totstu, label = "Students")
plt.plot(time,totallg, label = "Allergy")
plt.xlabel("time")
plt.ylabel("tot rev")
plt.legend()
plt.show()
```



✏️ *I made an allergy class that has a 30% chance that a person has allergies and the allergen people avoided that allergens in the vending machines and since all the candy is produced in the same facility they could all have peanuts so they avoid them in general, meaning it takes longer for the species with allergens to climb. *

✅ **Question 2.7 (1 point):** 🛑 **Pause to commit your work.** Save your notebook, then **add and commit** your changes using a message like `Committing Part 2` (you do not need to push yet, though you may). This point comes from your commit history showing a commit for Part 2.

Part 3: Analyze the machine layout as a graph (40 points)

Your simulation in Part 2 hinted at something the vending-machine operator really cares about: **where the machines are matters**. The machine nearest a crowd sold out first; a machine off on its own served an area no one else could reach. But a time-stepped simulation is a lot of machinery just to reason about **layout**. Sometimes the more useful question isn't "what happens minute by minute?" but simply **"how are these machines**

arranged relative to each other across campus?" -- which ones are clustered and redundant, which ones stand alone, and how the operator should stock them.

That kind of question is exactly what **graph theory** is good at. So in this part we set the simulation aside and model the machines as a **graph** (a network): each machine is placed at its approximate **MSU campus location** and becomes a **node**, and we draw an **edge** between two machines whenever they are within easy **walking distance** of each other. Turning positions into nodes-and-edges like this is the same move you made with the cell towers in the Day 11 activity -- and once the layout is a graph, a few lines of `networkx` can answer questions that would be awkward to get at any other way: *Which machine is the best-connected hub? Is any machine stranded with no neighbors (an under-served corner of campus)? How few distinct snack categories do we need so that neighboring machines never duplicate each other?* You'll answer each of those below.

> 💡 **Lean on `networkx`.** It's a library built specifically for network data, so most of the questions below have a **built-in function or method** -- you rarely need to compute things by hand. Keep the networkx documentation open and reach for a built-in whenever you can; part of the goal here is to see how much a purpose-built package gives you for free.

The positions below are **approximated from the campus map's grid** (`x` runs **west-to-east**, `y` runs **south-to-north**), and each machine is tagged with a rough campus `area`:

- **West:** Brody Hall, 1855 Place, Breslin Center
- **Central:** Spartan Stadium, Wells Hall, Main Library
- **North:** The Union, Broad Art Museum
- **East:** Shaw Hall, and -- way out in the East Neighborhood -- Akers Hall

> ⚠️ **These are still only rough coordinates, not an accurate map.** They are read off the map's grid in arbitrary "campus units" and capture the *relative* layout (roughly who is near whom), but they are **not** exact positions, true walking distances, or GPS coordinates -- so please don't use them to actually navigate campus!

If you want a visual reference of the campus layout, you can open the MSU campus map in a browser and compare the machine locations above to the real buildings.

In [87]:
```
## DO NOT MODIFY THIS CELL -- approximate MSU campus locations for the vending mach
# NOTE: positions are read ROUGHLY off the campus map grid (arbitrary "campus units
# x runs west->east, y runs south->north.   name -> (x, y, area)
machine_locations = {
    'Brody Hall':      (3.25, 7.5, 'West'),
    '1855 Place':      (3.5, 6.35, 'West'),
    'Breslin Center':  (4.0, 6.5, 'West'),
    'Spartan Stadium': (5.75, 6.5, 'Central'),
    'Wells Hall':      (6.25, 6.25, 'Central'),
    'Main Library':    (6.0, 7.25, 'Central'),
```

```
    'The Union':       (6.0, 8.5, 'North'),
    'Shaw Hall':       (7.75, 6.0, 'East'),
    'Akers Hall':      (10.25, 5.25, 'East'),
    'Broad Art Museum':(7.5, 8.0, 'North'),
}
```

✅ **Question 3.1 (5 points):** Build an **undirected** `networkx` `Graph` from `machine_locations`:

- Add one node per machine (use the **location name** as the node), storing its `(x, y)` position as a node attribute called `pos` (and, if you like, its campus `area`).
- Add an **edge** between any two machines whose straight-line distance is **less than or equal to 2.0** (our "walking distance" threshold, in campus units). The straight-line (Euclidean) distance between two points (x_1, y_1) and (x_2, y_2) is $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ -- and Python's `math.hypot(x2 - x1, y2 - y1)` computes exactly this in one call (remember to `import math`).

Print the number of nodes and edges.

Hint: when you add a node, can you attach extra information to it (like its position) at the same time? Check how `add_node` uses extra keyword arguments to accept attributes (e.g. `pos=(x, y)` or `area=area`).

In [44]:
```
# Put your code for Question 3.1 here.
import math
import networkx as nx

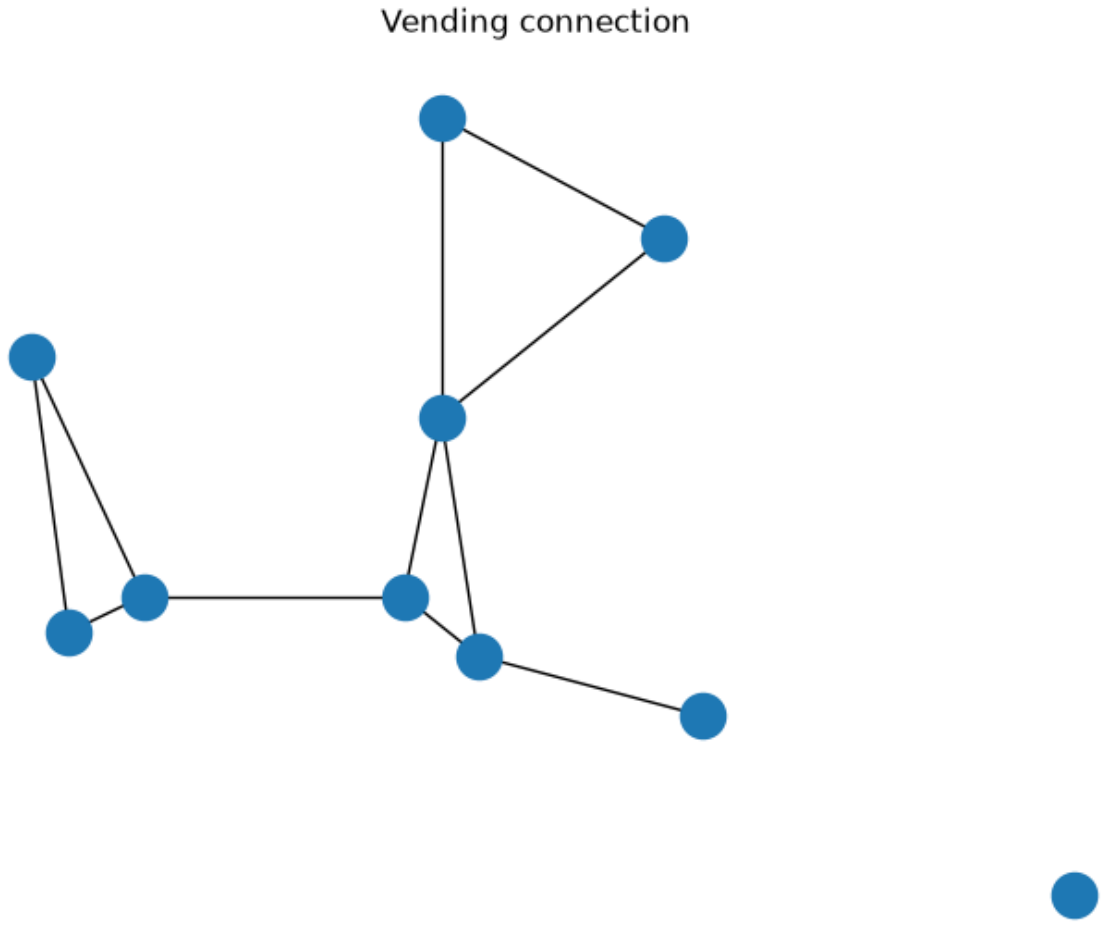
gr = nx.Graph()
for i in machine_locations:
    gr.add_node(i,pos = (machine_locations[i][0], machine_locations[i][1]), area =
for i in machine_locations:
    for j in machine_locations:
        if i != j:
            if math.dist(gr.nodes[i]['pos'],gr.nodes[j]['pos']) <= 2:
                gr.add_edge(i,j)
print(gr.number_of_nodes())
print(gr.number_of_edges())
```

```
10
11
```

✅ **Question 3.2 (4 points):** Draw the graph using the machines' real positions (use the `pos` attributes you stored). Give the figure a title.

Hint: does `networkx` have a function that draws a graph using a dictionary of node positions? And a helper method that collects a node attribute (you stored `pos`) from a graph object into exactly that kind of dictionary? Look these up in the docs rather than placing the nodes by hand.

In [45]:
```
# Put your code for Question 3.2 here.
import matplotlib.pyplot as plt
pos = nx.get_node_attributes(gr,'pos')
nx.draw(gr, pos=pos)
plt.title("Vending connection")
plt.show()
```



✅ **Question 3.3 (6 points):** Use your graph to answer the following (print your answers):

- How many machines (nodes) and connections (edges) are there?
- What is the **degree** of each machine, and which machine is the **most connected**?

- **Pick any one machine** and print its **neighbors** (the machines within walking distance of it).
- **Pick any one campus** `area` (`'West'`, `'North'`, `'Central'`, or `'East'`) and create the **induced subgraph** of just that area's machines; report how many **nodes** and **edges** it has.

Hint: `networkx` almost certainly has a built-in for each of these -- a good habit is to search the docs before writing a loop. Does a graph know how to report its own number of nodes? What about its number of edges? Is there a method to get a single node's degree, or to list its neighbors? And is there a way to pull out the subgraph induced by a chosen set of nodes? Search the networkx documentation for each.

Building the subgraph: the induced-subgraph method needs the **list of machine names** in your chosen area. Each machine's area is available in `machine_locations` (the third item in each tuple), and -- if you stored it in Q3.1 -- as a node attribute on the graph as well. Either way, build a list of the names whose area matches your pick, then hand that list to the subgraph method. (If you use the node attributes, check the docs for how to loop over nodes **together with their attribute data**.)

In [46]:
```
# Put your code for Question 3.3 here.
#10 nodes 11 edges
for i in gr.nodes:
    print(i,gr.degree(i))
#main library
mostcon = max(gr.nodes, key = gr.degree)
print(mostcon)
choice = "Spartan Stadium"
print(choice, list(gr.neighbors(choice)))
area = "Central"
central_node = []
for i in gr.nodes:
    if gr.nodes[i]["area"] == area:
        central_node.append(i)
sub = gr.subgraph(central_node)
print(area, sub.number_of_nodes())
print(area, sub.number_of_edges())
```

```
Brody Hall 2
1855 Place 2
Breslin Center 3
Spartan Stadium 3
Wells Hall 3
Main Library 4
The Union 2
Shaw Hall 1
Akers Hall 0
Broad Art Museum 2
Main Library
Spartan Stadium ['Breslin Center', 'Wells Hall', 'Main Library']
Central 3
Central 3
```

✅ **Question 3.4 (6 points):** The operator wants **adjacent** machines (ones connected by an edge) to stock **different** snack categories, so nearby machines are not redundant. This is a **graph coloring** problem -- exactly like the greedy coloring from the Day 11 activity.

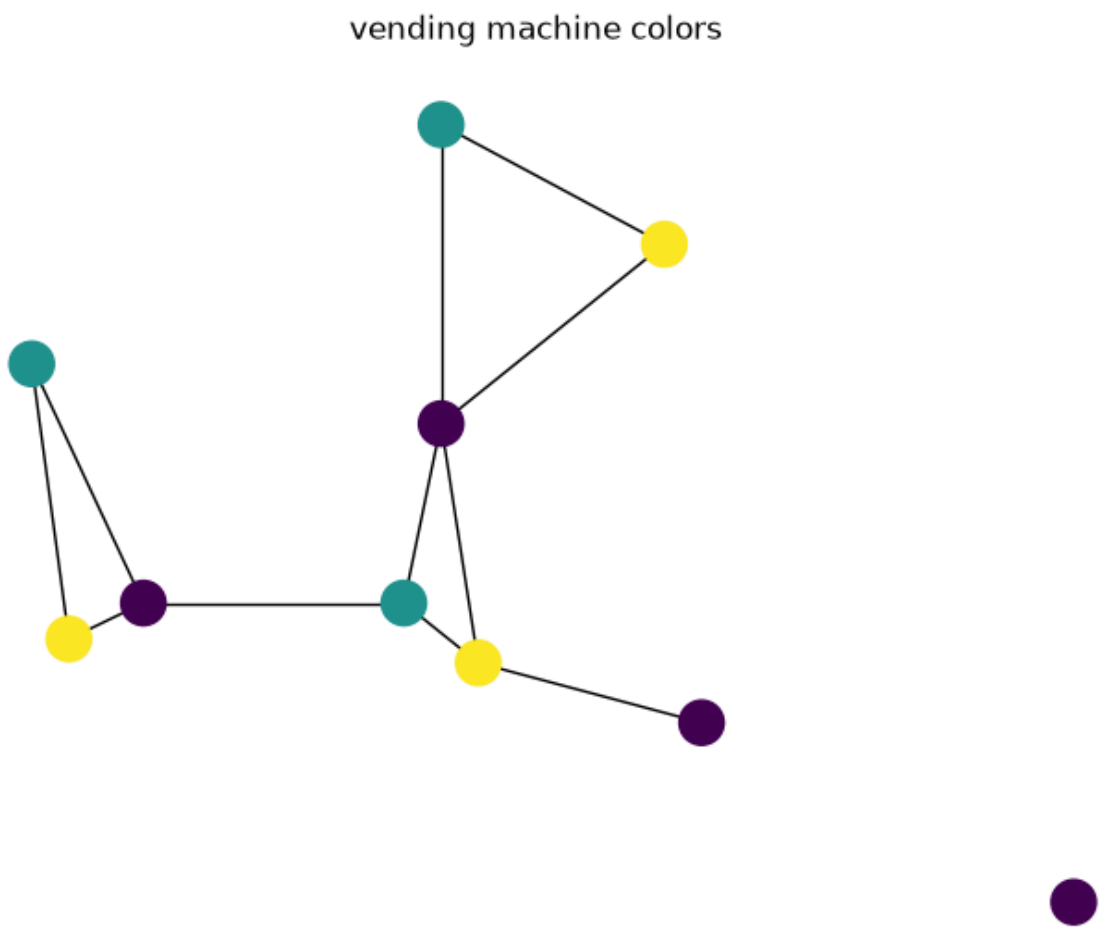
- Use `nx.greedy_color` to color the graph, and print the **minimum number of snack categories** this coloring needs.
- Then **draw the graph with each machine colored by its assigned category** (use the machine positions, as in Q3.2) so you can *see* that no two connected machines share a color.

In [47]:
```
# Put your code for Question 3.4 here.
import matplotlib.pyplot as plt
color = nx.greedy_color(gr)
num = max(color.values())+1
print(num)

pos = nx.get_node_attributes(gr, "pos")
colors = []
for i in gr.nodes:
    colors.append(color[i])
nx.draw(gr,pos=pos, node_color = colors)
plt.title("vending machine colors")
```

```
3
```

Out[47]:
```
Text(0.5, 1.0, 'vending machine colors')
```



✅ **Question 3.5 (4 points):** The graph you built depends entirely on the **walking-distance threshold** (you used `2.0`). Rebuild the graph using a **different threshold of your choice** (any value other than 2.0 -- larger or smaller). Print the new number of edges, and in the markdown cell describe **how the structure changed**: did more machines become connected? did the isolated machine gain a neighbor? did the most-connected machine change?

Answer this primarily by **inspecting the graph's properties directly** -- the edge count, the degrees, and which machines are now isolated -- the same way the earlier questions did. You're welcome to **also draw the new graph** if you'd like to *visually confirm* what those properties are telling you, but the drawing is **optional** and should not be your main way of reaching the answer.

In [57]:
```
# Put your code for Question 3.5 here -- rebuild with your own threshold and print
import math
import networkx as nx

grn = nx.Graph()

for i in machine_locations:
    grn.add_node(i,pos=(machine_locations[i][0], machine_locations[i][1]),area=mach
```

```
threshold = 3.0

for i in machine_locations:
    for j in machine_locations:
        if i != j:
            if math.dist(grn.nodes[i]["pos"], grn.nodes[j]["pos"]) <= threshold:
                grn.add_edge(i, j)
print(gr.number_of_edges())
print(grn.number_of_edges())
```

```
11
28
```

✏️ *When the threshold increased the number of nodes stayed the same but the edges increased from 11 to 28. The distance between the machines most likely stayed the same, but the number of connections between the machines increased substantially.

✅ **Question 3.6 (6 points):** In the cell below, **interpret** the graph analysis (based on the initial threshold of 2.0):

- Is any machine **isolated** (no neighbors)? What does that say about its placement/coverage?
- What does the graph coloring result mean in plain language for how the operator should stock the machines?
- Based on your analysis so far, give the operator **one** concrete recommendation about machine placement or stocking.

✏️ There is one isolated machine, as seen on the model, this means that it has no nearby machines, so it's out of the way if it runs out of snacks, and the student's that use it have to go out of there way as well. Placing another vending machine between like the library and east neighborhood would help connect even more machines and make restocking easier, or one by the buisness college. The graph coloring seperates 3 different machines, so the operator should stock based on the 3 different colors so no 2 vending machines are the same next to eachother.

✅ **Question 3.7 (8 points):** Your interpretation in Q3.6 probably pointed to a **coverage gap** (for example, an isolated or under-served machine). Now **act on it.** Choose a **real MSU location** that does not already have a machine, and add a **new** vending machine there to improve coverage:

- Give your new machine an approximate `(x, y)` position and an `area`, and add it as a node to a **copy** of the graph (do not modify the provided `machine_locations` cell).

- Add edges from your new machine to every existing machine within the walking-distance threshold (`2.0`).
- Print your new machine's **degree** and **neighbors**, and check whether the previously **isolated** machine (Akers Hall) is now connected.
- **Draw the modified graph** (using the machine positions) so you can *see* your new machine and the edges it adds.

Hint: work on a **copy** of your graph so you don't disturb the original -- does `networkx` give you a way to copy a graph? Add the new machine and its edges the same way you built the graph in Q3.1 (you can reuse the distance check), then inspect it with the same degree/neighbor methods you found in Q3.3.

Then, in the markdown cell, say **where you put your machine and why**, and whether it improved coverage (did it reach an under-served area, or just add to an already-crowded cluster?).

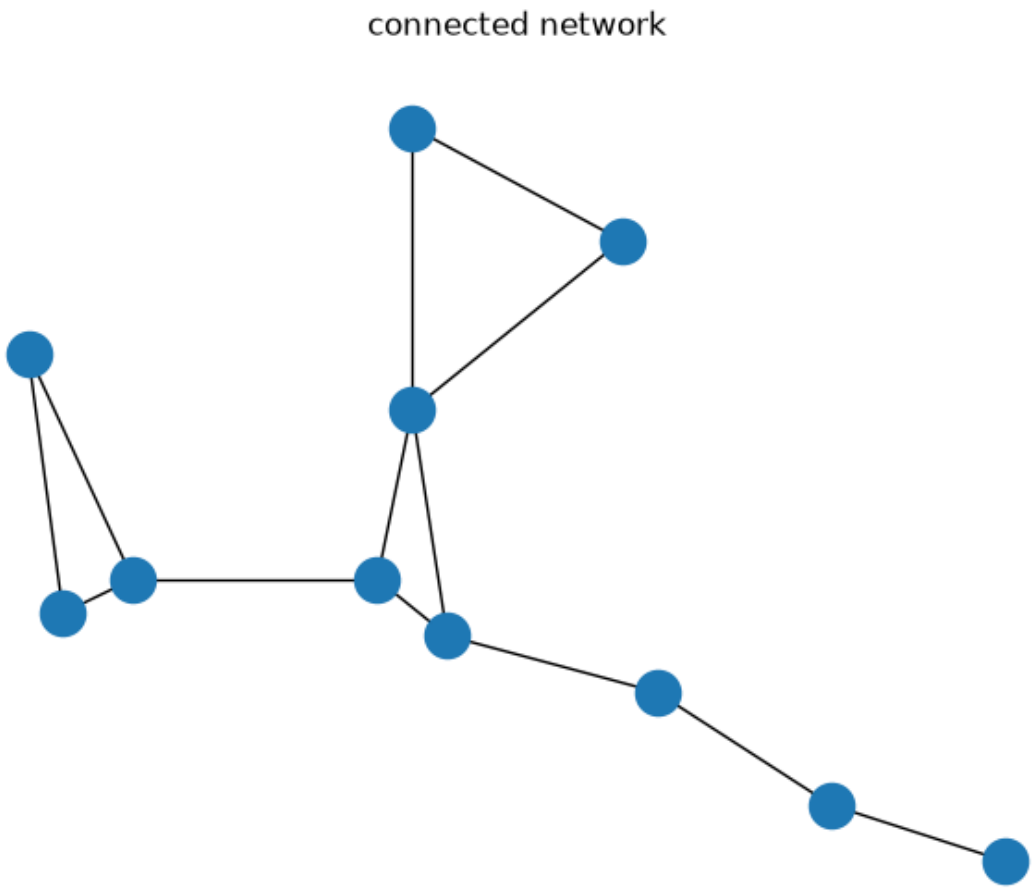
In [63]:
```
# Put your code for Question 3.6 here -- add your new machine and show how the grap
grmod = gr.copy()
newname = "Bus school"
newpos = (9.0, 5.5)
newarea = "East"
grmod.add_node(newname, pos=newpos, area=newarea)

for i in list(grmod.nodes):
    if i != newname:
        if math.dist(newpos, grmod.nodes[i]["pos"]) <= 2.0:
            grmod.add_edge(newname, i)

print(newname,grmod.degree(newname))
print(newname,list(grmod.neighbors(newname)))
print(grmod.degree("Akers Hall"))
print(list(grmod.neighbors("Akers Hall")))
pos = nx.get_node_attributes(grmod, "pos")
nx.draw(grmod, pos=pos)
plt.title("connected network")
```

```
Bus school 2
Bus school ['Shaw Hall', 'Akers Hall']
1
['Bus school']
```

Out[63]:
```
Text(0.5, 1.0, 'connected network')
```



✏️ *I put mine at the buisness school between akers and the last point, which is shaw. I also put it at the buisness school because it is in an acedemic building and students would get hungrier in an acedemic building and it also connects to the route better for distribution. This way all the points connect.*

✅ **Question 3.8 (1 point):** 🛑 **Pause to commit your work.** Save your notebook, then **add and commit** your changes using a message like `Committing Part 3` (you do not need to push yet, though you may). This point comes from your commit history showing a commit for Part 3.

Part 4: Reflection and AI/resource audit (10 points)

Now that you've built the model and analyzed the network, take some time to reflect on **how** you got there -- what you used, how you checked your work, and what you learned. Be candid: there's **no penalty** for having used AI or other resources, and transparency is part of

working professionally. (Course-policy reminder: AI tools are allowed *with attribution*; **undocumented** AI use is an integrity violation.)

✅ **Question 4.1 (4 points) -- Resource & AI audit:** Document the outside help you used on this assignment. For **each** resource you leaned on -- **HiTA or another AI tool**, the `networkx` / `pandas` /Python **documentation**, your **past assignments**, or **classmates** (within course policy) -- note:

- which **part/question** you used it for,
- **what** you used (for an AI tool, include the **actual prompt(s)** you gave it), and
- **how you used the result** -- did you take it as-is, modify it, or just use it to understand something?

A short bulleted list or a small table is fine. **If you did not use any outside resources, simply say so.**

Part	Tool	prompt	result	Changes
2.2	gemini	how do I call to the superclass in python?	class Parent: def greet(self): print("Hello from Parent") class Child(Parent): def greet(self): Parent.greet(self) print("Hello from Child")	My class was getting an error every time I tried to call the superclass with super, so I did actually change my code based on this.

2.3	gemini	Why is this code not implementing, it I get a "customer is not defined" def step(self):
		for i in self.customers:
		i.get_hungrier()
		if i.wants_to_buy():
		machine = self.nearest_machine(i)
		if len(machine.inventory) > 0 and i.cash >= machine.price:
		item = machine.inventory[0]
		
		machine.input_cash(machine.price)
		bought = machine.make_request(item)
		if machine.make_request(item):
		i.cash -= machine.price
		i.hunger = 0
		totrev = 0
		for j in self.machines:
		totrev = totrev + j.stash
		self.history.append(totrev)
			Line 6 has a bug where & is used instead of and:
			if len(machine.inventory) > 0 and i.cash >= machine.price:
			
			It looks like the issue you're encountering within the Customer class is most likely due to an execution order problem in your notebook environment. Specifically, Python evaluates classes and methods based on the current active state in memory, meaning that if the preceding cells—which define key dependencies, parent classes, or helper functions—haven't been executed yet in the current kernel session, Python won't recognize them when the Customer class attempts to run.
				I had to rerun my kernal because my session timed out and that was the only major issue.

2.4	gemini	why is build venue not defined in this line? venueallg = build_venue(base.machines, allg_customers)	build_venue is not defined for one of these reasons: 1. **Cell wasn't executed:** The cell defining def build_venue(...) hasn't been run in the current session. 2. **Missing import:** If it lives in an external module or separate .py file, you need to import it first (e.g., from filename import build_venue). 3. **Typo / Scope:** The name is misspelled, defined inside another class/function, or belongs to a class/module (e.g., base.build_venue(...) or Venue.build(...)). Run the cell containing the def build_venue definition first, or restart the kernel and run all cells from top to bottom.	None of these were the reason so this was really just useless, I had to change build_venue to just venue because venue was already made.

3.1	gemini	how does add_node work?	In NetworkX (and standard graph implementations), add_node inserts a single node into a graph:
			Python
			import networkx as nx
			
			G = nx.Graph()
			
			# Add a single node (can be an int, string, tuple, etc.)
			G.add_node(1)
			G.add_node("A")
				See problem below

			# Add a node with custom attributes/metadata
			G.add_node("B", color="red", weight=4.5)
			Key behaviors:
			• **Idempotent:** If the node already exists, calling add_node won't duplicate it. It simply updates any newly passed node attributes.
			• **Hashable types:** Any hashable Python object can be a node (numbers, strings, tuples), but mutable types like lists or dicts cannot.
			• **Adding multiple at once:** Use G.add_nodes_from([2, 3, 4]) or pass tuples with attribute dicts: G.add_nodes_from([(5, {"color": "blue"})]).	

3.1	gemini	what if I have multiple positional arguements like an x and y	**Pass a single identifier and set x and y as node attributes**
			Use a single label (like an ID, index, or name) as the positional argument, then pass x and y as keyword arguments:
			Python
			import networkx as nx
			
			G = nx.Graph()
			
			# Node label is 1, coordinates are attributes
			G.add_node(1, x=10, y=25)
				This one was actually helpful as a model and I used it.

			# Access the attributes print(G.nodes[1]) # {'x': 10, 'y': 25} print(G.nodes[1]['x']) # 10 Alternatively, you can pass them as a dictionary or tuple attribute: Python G.add_node("Machine_A", pos=(10, 25)) print(G.nodes["Machine_A"]["pos"]) # (10, 25)	
3.3	gemini	How does nx.draw work?	**nx.draw(G, pos=None, ax=None, **kwds) is a high-level wrapper around several lower-level drawing functions (draw_networkx_nodes, draw_networkx_edges, draw_networkx_labels).**	This was kind of helpful as a rough outline, this one was mostly to understand it conceptually though and it didn't do a good job.

✅ **Question 4.2 (2 points) -- Verifying your work:** Pick **two** of your answers from this assignment -- ideally **one from Part 2** (the ABM) and **one from Part 3** (the graph) -- and for each, explain **how you know it is correct**. Point to something concrete: a test cell that passed, a value you computed by hand and compared, a plot whose shape matched what you expected, or a graph property you cross-checked (e.g., the number of edges, or that a machine you thought was isolated really has degree 0). If you are *not* sure an answer is right, explain how you *would* check it.

✏️ *I know that my 2.3 passes because on the test slide it reads all tests pass, and I can tell that they do because all my code works for the rest of part 2 as well. Including my graphs which also make visual since being that the hunger rate climbs slower for visitors. For 3.2 I know my visualization is correct because the point's make since as a coordinate plane so it' easy to check, and also the next couple of questions are taylored around connecting the points so I know that there wasn't a connection to the rest of the vending machines which my graph also represented. *

✅ **Question 4.3 (3 points) -- Reflecting on your process:** In a few sentences each:

- **(1 pt)** Where did an outside resource or AI tool **genuinely help** you, and where did you rely mostly on **your own understanding**?

- **(1 pt)** Which half was more challenging for you -- **building the agent-based model (Part 2)** or **analyzing the graph (Part 3)** -- and **why**?
- **(1 pt)** This assignment leaned on purpose-built libraries (`networkx` , `pandas`). Name **one thing you now understand better** about using a library designed for a job versus writing everything yourself from scratch.

✏️ *The Ai tool helped when I needed to like proofread something really quick, I think my second use was a good representation where it told me to run my code again and replace & with and, which is bigenough to make it not run but small enough to not catch my eye. Part 3 most definetly, I have java experience so working through classes takes time, but I can figure everything out, but the graph part confused me until the later parts. I learned the types of arguements that are required for networkx, there were a lot of unfamiliar parts likt add_node that confused me but I understood them better now then I did earlier.*

✅ **Question 4.4 (1 point):** 🛑 **Final commit and push.** Save your notebook, then **add, commit, and push** your completed assignment to GitHub. This point comes from your final commit being pushed to your `cmse202-f26-turnin` repository.

Assignment wrap-up

Congratulations, you're done! Make sure you have **saved, committed, and pushed** your final notebook to your `cmse202-f26-turnin` repository, and submit this assignment by uploading it to the course **GradeScope** page, which you can find by using the dropdown menu on the right side of the navigation bar in D2L. Look for **"Homework 02 - Student Notebook Submission"** assignment and upload your Jupyter Notebook there -- once it uploads, double-check that you submitted the version with your completed solutions! You should also get a confirmation that a PDF version of your submission was successfully built. We will grade the PDF version.

For the full step-by-step process, see the Gradescope Submission Instructions on the course website. **It is important that you complete "Step 3" of the process and tag your solutions to each specific question of the assignment outline.**

© Copyright 2026, Department of Computational Mathematics, Science and Engineering at Michigan State University